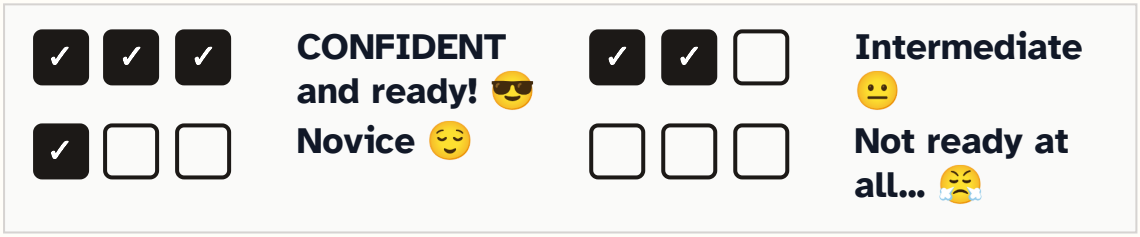
FIT3155 Exam Checklist



Exam facts: 60 marks. 10 questions, 6 marks each. Weeks 1-11 only.

Strategy: attempt ALL questions, confident ones first, never sink 20 min into one question for 3 extra marks.

Difficulty tags: 🟢 free marks, 🟡 medium, 🔴 the hard ones

W1: Z-Algorithm 🟢

☐☐☐ **Z-value definitions**

- $Z_i(S)$, Z-box, l and r pointers

☐☐☐ **The cases**

- Case 1 ($k > r$): explicit comparison from scratch
- Case 2a ($Z[k-l+1] < r-k+1$): copy, no comparison
- Case 2b ($Z[k-l+1] \geq r-k+1$): box may extend past r
  - 2b.1 ($>$): $Z[k] = r-k+1$ exactly, no comparison
  - 2b.2 ($=$): compare past r to extend

☐☐☐ **Why $O(n)$**

- r only ever moves right
- Each comparison either matches (extends r) or mismatches (ends extension)

☐☐☐ **Exact pattern matching**

- pat$txt concatenation, $O(m+n)$

☐☐☐ **Hand-trace a Z-array, labelling the case at each k**

☐☐☐ **Exam style (sample Q1): use `zalg` as a black box on a fresh problem**

- Periods, borders, rotations
- Longest suffix of S matching a prefix of T
- Z_{suffix} array (match suffix instead of prefix)
- MP / matched-prefix array (largest suffix of $S[i..N]$ = prefix of S)

W2: Boyer-Moore 🟡

- [ ] **Right-to-left scanning**
- [ ] **Bad character rule**
  - Basic rule and shift amount
  - Extended: per-position rightmost-occurrence tables
  - Construction time/space + retrieval time of each
  - $O(m + |\Sigma|)$ space-optimised extended table vs the $m \times |\Sigma|$ table
- [ ] **Good suffix rule**
  - gs array from Z-values of the reversed pattern
  - Matched prefix rule (mp array) when $gs[k+1] = 0$
- [ ] **Combined shift = $\max$(bad character, good suffix / matched prefix)**
- [ ] **Galil's optimisation**
  - Stop/start boundaries, gives linear time
  - Count explicit comparisons on periodic text/pattern
- [ ] **Exam style (sample Q2): PROVE a shift is safe (three variants)**
  - gs rule when $gs[k+1] > 0$: shift $m - gs[k+1]$
  - matched-prefix rule when $gs[k+1] = 0$: shift $m - mp[k+1]$
  - special $mp[2]$ after a FULL match: shift $m - mp[2]$
  - Contradiction: a skipped occurrence would force a bigger gs/Z value
- [ ] **Hand-trace BM showing each shift and which rule won**
- [ ] **KMP failure function (optional)**

W3: Burrows-Wheeler Transform 🟡

- [ ] **BWT construction**
  - Cyclic rotation matrix M, BWT = last column L
  - Via suffix array: $\mathrm{BWT}[i] = S[\mathrm{SA}[i] - 1]$
- [ ] **F column and rank**
  - F from sorting L
  - $\mathrm{rank}(c, i)$ / occurrence counts
- [ ] **LF mapping**
  - j-th c in L is the j-th c in F, and why
  - Formula: $p = \mathrm{rank}(L[i]) + \mathrm{nOccurrences}(L[i] \text{ in } L[1..i])$, and its proof
- [ ] **Invert a BWT by hand**
- [ ] **Backward search**
  - Pattern matching in time proportional to pattern length
  - sp/ep update rules and their correctness proof
  - Number of occurrences ($ep - sp + 1$) vs loci (needs suffix array)
- [ ] **Exam style (sample Q3): pseudocode the rank/occurrence structure**
  - State worst-case time AND space of construction
  - Two distinct nOccurrences structures: one for inversion, one for search
- [ ] **Why BWT compresses well (runs)**

W4: Ukkonen's Suffix Tree 🔴

- [ ] **Suffix tree basics**
  - Edge-label compression (start, end)
  - Internal nodes have ≥ 2 children, $ terminal
- [ ] **Implicit suffix trees, phases i, extensions j**
- [ ] **The three rules**
  - Rule 1: leaf extension
  - Rule 2 (regular): branch + new internal node
  - Rule 2 (alternate): branch at an existing node, no new node
  - Rule 3: already inside, do nothing
- [ ] **Suffix links**
  - Which nodes get them, when created, how traversal uses them
- [ ] **Skip-count trick**
- [ ] **Rapid leaf extension**
  - $globalEnd$ pointer
  - Once a leaf, always a leaf
- [ ] **Showstopper rule**
  - First Rule 3 ends the phase
  - Remember the active point for next phase
- [ ] $O(n)$ **proof**
  - $lastj$ never decreases, active point movement amortizes
- [ ] **Exam style (sample Q4): yes/no + justify on rule interactions**
  - Can Rule 2 fire in the LAST extension of a phase?
  - After Rule 3 at extension j in phase i, can j use Rule 1 in phase $i+1$?
- [ ] **Suffix tree applications (assume tree precomputed)**
  - Count occurrences of a substring; list all its positions
  - Longest repeated substring
  - Smallest substring occurring exactly k times
  - Lexicographically smallest suffix
- [ ] **Suffix tree $\rightarrow$ suffix array and BWT in linear time**
- [ ] **Hand-trace Ukkonen, labelling the rule at every extension**

W5: Data Compression 🟢

☐☐☐ **Information and entropy**

- $I(x) = -\log_2 P(x)$
- Entropy H = expected information

☐☐☐ **Codes**

- Fixed vs variable length
- Prefix-free property and why decoding needs it

☐☐☐ **Huffman encode/decode by hand**

☐☐☐ **Huffman code properties**

- Total bits = $\sum_i f_i \cdot l_i$
- Min/max possible codeword length; height $\geq \lceil \log_2 n \rceil$
- More frequent symbols are never longer-coded; the two least-frequent are deepest sibling leaves (longest codewords)

☐☐☐ **Minimum-variance Huffman (break ties by smallest subtree height)**

☐☐☐ **Elias omega**

- Encode by hand (recursive length-of-length prefixes)
- Decode a bitstream by hand
- When the number of length-components increases (powers of 2)

☐☐☐ **LZ77**

- Sliding window, [offset, length, next char] triples
- Encode AND decode by hand

☐☐☐ **Exam style (sample Q5): Elias omega codeword + decode LZ77 tuples**

☐☐☐ **LZSS variant**

- Flag bits, when a literal beats a reference

☐☐☐ **A real format combining Huffman + Elias + LZ**

W6: Semi-Numerical Algorithms 🟡

- [ ] **Karatsuba**
  - 3 half-size products instead of 4
  - $T(n) = 3T(n/2) + O(n) = O(n^{\log_2 3})$
  - Solve the recurrence by back-substitution, NOT Master Theorem
- [ ] **Modular arithmetic identities for +, -, * under mod**
- [ ] **Modular exponentiation by repeated squaring**
  - $O(\log b)$ multiplications via binary expansion of exponent
  - LSB-to-MSB (standard) and MSB-to-LSB (left-to-right) variants
- [ ] **Naive primality testing**
  - Trial division to $\sqrt{n}$, exponential in input BITS
- [ ] **Fermat**
  - Fermat's little theorem
  - Fermat test, broken by Carmichael numbers
- [ ] **Square roots of unity**
  - $x^2 \equiv 1$ (mod prime p) forces $x = \pm 1$
- [ ] **Exam style (sample Q6): describe ALL Miller-Rabin steps from memory**
  - Write $n - 1 = 2^s \cdot t$ (t odd)
  - The squaring sequence, witnesses, accept/reject logic
- [ ] **Error probability per trial, amplification over k trials, complexity**
- [ ] **Hand-run Miller-Rabin on a small n with given base a**

W7: Fibonacci Heaps 🔴

- [ ] **Structure**
  - Circular doubly-linked root list, min pointer
  - Child lists, degree, mark bits
- [ ] **Lazy operations: find-min $O(1)$, insert $O(1)$**
- [ ] **Exam style (sample Q7): pseudocode `merge` in $O(1)$**
  - Concatenate root lists, update min
- [ ] **extract-min**
  - Promote children to root list
  - Consolidate: link equal-degree trees
  - Find new min
- [ ] **decrease-key**
  - Cut to root list
  - Cascading cuts of marked ancestors
- [ ] **delete = decrease-key to $-\infty$ + extract-min**
- [ ] **Why mark nodes**
  - Each non-root loses at most 1 child, keeps trees bushy
- [ ] **Max degree bound**
  - Degree-k subtree has $\geq F_{k+2}$ nodes $\rightarrow$ max degree $O(\log n)$
  - Underpinning proofs (by induction): $\sum_{i=0}^{n} F_i = F_{n+2} - 1$
  - and $F_{n+2} \geq \phi^n$ (golden ratio growth)
- [ ] **Amortized costs via $\Phi = \text{trees} + 2 \cdot \text{marks}$ (links to W8)**
- [ ] **Dijkstra speedup: $O(E + V \log V)$**
  - General form: $|V| \cdot T_{\text{extract-min}} + |E| \cdot T_{\text{decrease-key}}$
- [ ] **Hand-trace: inserts, extract-min consolidation, cascading cut**

W8: Amortized Analysis 🟡🔴

☐☐☐ **What amortized means**

- vs worst-case per operation
- Valid over any sequence from an empty structure

☐☐☐ **Aggregate method**

- Bound total cost of n ops, divide by n

☐☐☐ **Accounting method**

- Assign credits, stored credit never goes negative

☐☐☐ **Potential method**

- $\Phi(D_0) = 0, \Phi(D_i) \geq 0$
- Amortized = actual + $\Phi(D_i) - \Phi(D_{i-1})$

☐☐☐ **Exam style (sample Q8): binary counter increment via AGGREGATE**

- Bit i flips $n/2^i$ times, total $< 2n \rightarrow O(1)$ amortized

☐☐☐ **Binary counter via accounting AND potential too**

☐☐☐ **Dynamic array doubling via all three methods**

- Potential $\Phi = 2 \cdot \text{size} - \text{capacity}$

☐☐☐ **Fibonacci heap potential analysis**

- extract-min $O(\log n)$, decrease-key $O(1)$ amortized

☐☐☐ **Designing a potential/credit scheme for an UNSEEN structure**

W9: B-Trees 🟢

☐☐☐ **Properties**

- Minimum degree t
- Non-root: keys in $[t-1, 2t-1]$, children in $[t, 2t]$
- Root exception: may have fewer (≥ 1 key, ≥ 2 children if internal)
- All leaves at same depth

☐☐☐ **Search: $O(\log_t N)$ node visits**

- Pseudocode $\text{search}(P, x)$ + justify termination

☐☐☐ **Insert with PROACTIVE splitting**

- Split any full node on the way down, one pass

☐☐☐ **Delete cases**

- Leaf, internal (swap predecessor/successor)
- Borrow from sibling, merge with sibling

☐☐☐ **Exam style (sample Q9): derive height bounds**

- Upper: $h \leq \log_t((N+1)/2)$ from minimum node counts per level
- Lower bound on h as a function of t and N (max node counts)

☐☐☐ **Why B-trees suit disks (one node = one block)**

☐☐☐ **Can all nodes be simultaneously full, or all at min occupancy?**

☐☐☐ **Worst-case space $O(N)$; search/insert/delete complexity**

- $O(\log_t N)$ node/disk visits
- $O(\log N)$ comparisons (binary search inside each node)
- $O(t \log_t N)$ if in-node scanning/moving is counted

☐☐☐ **Amortised split/merge counts over a sequence**

W10: Linear Programming and Simplex 🔴

☐☐☐ **LP ingredients**

- Decision variables, linear objective, linear constraints

☐☐☐ **Standard/canonical form**

- Converting $\geq$ and $=$ constraints

☐☐☐ **Slack variables and basic solutions**

- Basic vs non-basic variables
- Basic feasible solution = vertex of feasible region
- Why the optimum lives at a vertex
- Search space = $\binom{N+m}{m}$ basic solutions; vertex enumeration is exponential

☐☐☐ **Algebraic simplex**

- Entering variable: improves z
- Leaving variable: min-ratio test

☐☐☐ **Tableau simplex**

- Set up tableau, pivot row operations, read off solution
- Optimality: no improving coefficient left in objective row
- Pseudocode the tableau method (inputs c, B, rhs)
- Unique optimum vs alternative optima (zero reduced cost on a non-basic var)

☐☐☐ **Exam style (sample Q10): SOLVE a small max LP by tableau, start to finish**

☐☐☐ **Formulating a word problem as an LP (lab style)**

W11: Hungarian Algorithm 🟡

The problem

- Min-cost perfect matching / linear assignment, $N \times N$ cost matrix
- Matching, maximum-cardinality, perfect matching definitions
- ILP formulation with binary x_{ij} decision variables

Problem variants (lab style)

- Max-weight $\rightarrow$ min-cost conversion (subtract entries from the max)
- Rectangular/unbalanced matrix: pad with dummy zero row/column

Key invariance

- Adding a constant to any row/column never changes the optimal assignment

Dual variables

- u_i, v_j with feasibility $u_i + v_j \leq c_{ij}$
- Reduced costs $c'_{ij} = c_{ij} - u_i - v_j \geq 0$

The reduction steps

- Step 1: row reduction (u_i = row minimum)
- Step 2: column reduction (v_j = column minimum)
- Grow assignment on zeros; when stuck, update u_i, v_j by minimum uncovered value

Termination and optimality

- Perfect matching among zeros $\rightarrow$ optimal, cost = $\sum u_i + \sum v_j$
- Dual is a lower bound on every matching, equality achieved (cf. max-flow = min-cut)

Hand-run the full reduction on a 3×3 to 5×5 matrix

Complexity: $O(N^4)$ originally, $O(N^3)$ refined